

Investigating Code Similarity Patterns in LLM-Generated and Human-Written Programming Solutions

Paulina Gacek^a, Bartosz Gdowski^b, Konrad Szymański^c and Wojciech Żmuda^d
AGH University of Krakow, Faculty of Electrical Engineering, Automatics, IT and Biomedical Engineering, Poland

Keywords: Large Language Models, Code Similarity, Computer Science Education, Code Generation, Plagiarism Detection, Source Code Plagiarism.

Abstract: As Large Language Models (LLMs) reshape programming education, detecting AI-generated submissions using traditional similarity-based plagiarism tools presents a novel challenge. This paper investigates whether LLM-generated code exhibits structural and lexical patterns that systematically differ from human-written solutions. Analyzing a dataset of 9,000 solutions to constrained algorithmic tasks, we compare pre-2023 human submissions against synthetic outputs from GPT-5.2 and Gemini 3.0 Flash. Our findings reveal that AI solutions occupy a highly concentrated space, exhibiting extreme structural convergence (75% similarity compared to 28% for humans) and highly constrained lexical overlap. While commenting behaviors vary significantly by model, these overarching convergences suggest similarity-based detection is theoretically viable. However, due to severe risks of model-specific statistical masking and false positives, strict pedagogical precautions are required before deployment.

1 INTRODUCTION

The rapid adoption of large language models (LLMs) has fundamentally altered programming education. While widespread student use of generative AI (Dzialets et al., 2025; Li et al., 2025; Gacek, 2025) can support learning, it introduces significant assessment and integrity challenges. The primary concern is not AI usage itself, but its potential to bypass foundational learning, allowing students to submit correct solutions without grasping the underlying logic.

Traditional plagiarism detection systems rely on measuring similarity between student submissions (Schleimer et al., 2003; Prechelt and Malpohl, 2003; Yan et al., 2018). However, such tools were designed to identify copied or collaboratively produced code. LLM-assisted generation represents a different phenomenon: students may submit syntactically correct and functionally valid programs that were generated independently by an AI system rather than copied from peers. Detecting such cases is difficult, particularly when instructors have access only to

final submissions.

A central open question is whether LLM-generated solutions exhibit structural patterns that systematically differ from human-written solutions. Due to the semi-deterministic nature of autoregressive models, LLM outputs may form more concentrated or homogeneous solution distributions (Cotroneo et al., 2025). If this is the case, similarity-based analysis could provide insights into AI-assisted code generation. Conversely, if LLM-generated solutions are as diverse as human ones, detection based on similarity alone would be insufficient.

This paper investigates similarity patterns in human-written and LLM-generated solutions to algorithmic programming problems. Drawing on a dataset constructed from community-sourced submissions and their LLM-generated counterparts, the analysis evaluates intra-group and cross-group similarity at both lexical and structural levels. The scope of this study is deliberately constrained to algorithmic tasks. While utilizing LLMs for complex, multi-file software engineering projects generally requires an underlying structural understanding from the user, introductory algorithmic problems represent a uniquely vulnerable environment. In these constrained spaces, students can easily employ zero-shot generation—simply inputting a problem descrip-

^a <https://orcid.org/0009-0008-1242-7542>

^b <https://orcid.org/0009-0004-8159-461X>

^c <https://orcid.org/0009-0007-1831-9676>

^d <https://orcid.org/0009-0003-6393-4732>

tion to achieve a maximum grade with zero cognitive effort or comprehension. Rather than proposing a production-ready detection system, the primary objective is to determine whether measurable differences in solution distributions exist within these high-risk environments and to explore their implications for AI-aware assessment in programming education.

2 RELATED WORK

Plagiarism Detection in Programming Education. Similarity-based plagiarism detection has long been employed in programming education to identify excessive collaboration and copied assignments. Tools such as MOSS (Schleimer et al., 2003) and JPlag (Prechelt and Malpohl, 2003) compare student submissions by analyzing lexical and structural similarity. TMOSS (Yan et al., 2018) extends this approach by incorporating intermediate development snapshots, enabling earlier detection of problematic collaboration patterns. However, in typical educational settings, instructors only receive final submissions, as commonly used learning management systems and grading workflows do not capture intermediate development states. This infrastructural limitation significantly reduces the practical applicability of approaches that rely on incremental monitoring.

Prior research indicates that academic misconduct in programming is often situational rather than purely malicious (Sağlam, 2025). Students who engage in excessive collaboration tend to demonstrate reduced engagement and poorer academic outcomes (Yan et al., 2018). These findings underscore the importance of detection approaches that are not only technically robust but also pedagogically sound.

AI-Assisted Code Generation and Detection. The rapid adoption of large language models (LLMs) has introduced new challenges for academic integrity in programming education. Surveys report widespread student use of generative AI tools for solving programming tasks (Gacek, 2025; Li et al., 2025). Unlike traditional plagiarism, AI-assisted generation may involve producing an entire solution directly from the assignment specification, without copying peer submissions. Such scenarios severely complicate similarity-based detection approaches.

Recent studies have evaluated the effectiveness of AI-generated content detectors for source code (Suh et al., 2024). These findings suggest that current detection mechanisms struggle to reliably distinguish between human-written and AI-generated programs, particularly as models evolve rapidly.

Ellis et al. (Ellis et al., 2024) evaluated ChatGPT-generated solutions to introductory Python assignments, focusing on grading outcomes and instructor detection. While their findings showed ChatGPT performing at a mid-range student level, the study's reliance on an early text-only version of the model (Ye et al., 2023) introduces a significant modality misalignment. Majority of the evaluated tasks involved graphical programming (drawing), yet the model was constrained to text-based reasoning without the ability to iterate upon the visual output it produced. This lack of visual feedback likely affected task interpretation and the refinement of spatial logic in the code. In contrast, the present study shifts focus from these early modality constraints to investigate the underlying structural similarity distributions within and across human and LLM-generated solution sets.

Stylometry and Code Authorship Attribution. Stylometric approaches assume that authors exhibit relatively stable stylistic and structural patterns that can serve as identifying characteristics. Ambati et al. (Ambati et al., 2023) established a baseline for attributing AI-generated code using stylometric features. However, their work predates the widespread integration of advanced coding assistants and focuses on competitive programming datasets such as Google Code Jam (Zibada, 2023). Competitive programming submissions are typically concise and optimized for speed, which differs fundamentally from the structured assignments common in university education. Moreover, prior stylometric studies often assume AI generation is conditioned on partially written human code. In contrast, a pressing concern in foundational programming courses involves the zero-shot generation of complete solutions. The structural characteristics of such outputs may differ substantially from previously examined scenarios.

Comparative Studies of Human and AI-Generated Code. Recent large-scale empirical studies have compared human-written and AI-generated code using static analysis tools to assess defect rates, security vulnerabilities, and structural complexity (Cotroneo et al., 2025). These works report that AI-generated code often exhibits concise and pattern-based structures. While such studies provide valuable insights into software quality and complexity, they evaluate code artifacts through static quality metrics rather than similarity distributions across solutions. Furthermore, they primarily analyze large-scale professional or open-source corpora.

In contrast, this research focuses on short, algorithmic programming tasks typical of university as-

signments. These tasks are characterized by constrained solution spaces and limited architectural variability, providing a distinct and educationally relevant setting for investigating similarity patterns. Rather than assessing defects or vulnerabilities, this study employs established plagiarism detection and similarity analysis tools to examine intra-group and cross-group similarity among human-written and LLM-generated solutions to identical problems.

Recent studies have begun to examine structural and stylistic differences between human and AI-generated code (Shi et al., 2025; Bisztray et al., 2026), as well as detection approaches based on synthetic AI-generated references (Bashir, 2025). However, systematic analysis of similarity distributions across independently generated zero-shot solutions in constrained educational settings remains limited.

3 METHODOLOGY

The primary objective of this study is to investigate measurable similarity patterns across human-written and LLM-generated solution sets to enable more robust detection of AI-authored code. To evaluate these patterns, a systematic collection and analysis of algorithmic programming solutions was conducted. Because AI models can readily mimic human code at the surface level, this methodology moves beyond basic text-based comparisons. Instead, it employs a multi-dimensional approach, evaluating both the underlying structural convergence and the lexical diversity of the solution distributions.

3.1 Dataset Construction

The dataset utilized in this study consists of 100 algorithmic programming tasks classified as *Medium* difficulty from the LeetCode¹ platform. Medium-difficulty tasks were specifically chosen to ensure non-trivial solution spaces that require multi-line implementations and algorithmic reasoning beyond basic control flow. Tasks at the extremes of the difficulty spectrum were deliberately excluded to maintain educational relevance and analytical integrity. *Easy* problems were omitted due to their limited structural variability, which tends to artificially inflate baseline similarity measures. Conversely, *Hard* problems were excluded because they frequently demand highly specialized, non-obvious algorithmic insights or extreme optimizations. Such complex tasks are generally unrepresentative of foundational university-

level programming assignments and could introduce confounding variables related to task comprehension rather than natural stylistic convergence.

The selected tasks span multiple algorithmic paradigms, including dynamic programming, graph traversal, and greedy algorithms, to mitigate bias toward any single solution pattern.

3.1.1 Human Solutions

Human-generated solutions were collected programmatically using LeetCode’s GraphQL interface. For each task, Python solutions sorted by vote count were retrieved to prioritize high-quality community submissions. To ensure a clean baseline, only submissions created on or before November 30, 2022, were included. This cutoff precedes the public release of ChatGPT (OpenAI, 2022), mitigating the likelihood of AI-contaminated human samples. For each task, 50 solutions were retained to capture natural convergence toward canonical strategies.

3.1.2 LLM-Generated Solutions

Synthetic solutions were generated using two widely adopted contemporary large language models: GPT-5.2 (OpenAI, 2026) and Gemini 3.0 Flash (Google DeepMind, 2026). Both models were accessed with default inference settings to simulate typical student usage. For each model and task, 20 independent solutions were generated via independent zero-shot prompts without iterative refinement. The prompt template provided the original problem description and a Python code scaffold, instructing the model to return executable code only.

3.1.3 Final Dataset Composition

The final dataset comprises 9,000 total solutions across 100 tasks, distributed as follows: 5,000 human solutions, 2,000 GPT-generated solutions, and 2,000 Gemini-generated solutions. This structure enables robust intra-group and cross-group similarity comparisons.

3.2 Analytical Metrics

Let T represent the set of all algorithmic programming tasks evaluated in this study. For any given task $t \in T$, let $P(t)$ denote the set of all associated solution programs. An individual solution is thus denoted as $p \in P(t)$. To comprehensively evaluate the convergence of these solutions, similarity is quantified across two representational layers: structural and lexical.

¹<https://leetcode.com/>

3.2.1 Structural Similarity

To evaluate structural similarity independent of lexical and formatting variation, each solution p is parsed into an Abstract Syntax Tree (AST), denoted as T_p . To isolate control-flow and structural organization, all identifiers and literal values are normalized by replacing them with canonical placeholders (e.g., standardizing all variable names to `VAR` and literals to `VAL`), yielding a normalized tree, $\text{norm}(T_p)$.

Instead of computing exact Tree Edit Distance, which is computationally expensive for large-scale pairwise comparisons, structural similarity is approximated using subtree hashing. The normalized AST is recursively decomposed into a set of all possible subtrees, $S(\text{norm}(T_p))$. A canonical hash function H is then applied to every subtree $s \in S(\text{norm}(T_p))$ based on its node type and child structure. Thus, a solution p is represented by a set of subtree fingerprints, $F(p)$, defined as:

$$F(p) = \{H(s) \mid s \in S(\text{norm}(T_p))\} \quad (1)$$

The structural similarity between two distinct solutions for the same task, $p_i, p_j \in P(t)$, is then computed as the Jaccard similarity coefficient of their respective fingerprint sets:

$$J(p_i, p_j) = \frac{|F(p_i) \cap F(p_j)|}{|F(p_i) \cup F(p_j)|} \in [0, 1] \quad (2)$$

This representation captures control-flow structure, nesting patterns, and algorithmic organization while remaining computationally scalable across thousands of intra-task pairwise comparisons.

3.2.2 Lexical Similarity

Lexical characteristics are analyzed at multiple levels to capture stylistic and naming conventions:

Comment Similarity Analysis. As natural language comments are not strictly bound by syntax or algorithmic correctness, they serve as a high-entropy signal for stylistic regularity. Let $\text{com}(p)$ denote an extraction function that isolates and concatenates all natural language comments from a given solution $p \in P(t)$. To quantify surface-level repetition and stylistic convergence, the extracted textual content is mapped to a Term Frequency-Inverse Document Frequency (TF-IDF) vector space.

Let $\mathbf{v}_c(p) = \mathbf{v}(\text{com}(p)) \in \mathbb{R}^d$ denote the resulting TF-IDF vector representation of the extracted comments over a task-specific vocabulary of size d . The lexical similarity of comments between two distinct solutions, $p_i, p_j \in P(t)$, is then computed via cosine

similarity:

$$\text{Sim}_{\cos}(p_i, p_j) = \frac{\mathbf{v}_c(p_i) \cdot \mathbf{v}_c(p_j)}{\|\mathbf{v}_c(p_i)\| \|\mathbf{v}_c(p_j)\|} \in [0, 1] \quad (3)$$

Identifier Diversity. To quantify naming variability, Shannon entropy is computed over user-defined identifiers within each solution. Let $I(p)$ represent the set of all unique user-defined identifiers in solution p , and let $\text{Prob}(id)$ represent the probability of occurrence for a specific identifier $id \in I(p)$, calculated as its relative frequency within the program. The identifier entropy, $H(p)$, is defined as:

$$H(p) = - \sum_{id \in I(p)} \text{Prob}(id) \log_2 \text{Prob}(id) \quad (4)$$

Higher entropy ($H(p) \gg 0$) indicates greater unpredictability and lexical diversity within p , while lower entropy reflects convergence toward a smaller, more canonical vocabulary. Furthermore, cross-solution identifier similarity between $p_i, p_j \in P(t)$ is measured using the Jaccard overlap of their respective identifier sets, denoted as J_{id} :

$$J_{id}(p_i, p_j) = \frac{|I(p_i) \cap I(p_j)|}{|I(p_i) \cup I(p_j)|} \in [0, 1] \quad (5)$$

This metric captures the extent to which independent solutions reuse the exact same naming conventions for a given task.

4 RESULTS

4.1 Structural Convergence Analysis

Figure 1 presents the distribution of structural similarity scores computed via normalized AST subtree hashing. To highlight the extremes of this structural convergence, Table 1 summarizes the proportion of intra- and cross-group pairwise comparisons that exceed critical similarity thresholds.

Table 1: High-similarity structural overlap across solution sources. Percentages denote the proportion of pairwise comparisons exceeding specified similarity thresholds.

Comparison	>0.60	>0.90	= 1.00
Human–Human	1.56%	0.26%	0.24%
GPT–GPT	82.32%	49.05%	48.47%
Gemini–Gemini	74.49%	34.88%	34.83%
Human–GPT	2.16%	0.36%	0.29%
Human–Gemini	3.91%	0.45%	0.40%
Human–AI	3.04%	0.41%	0.34%

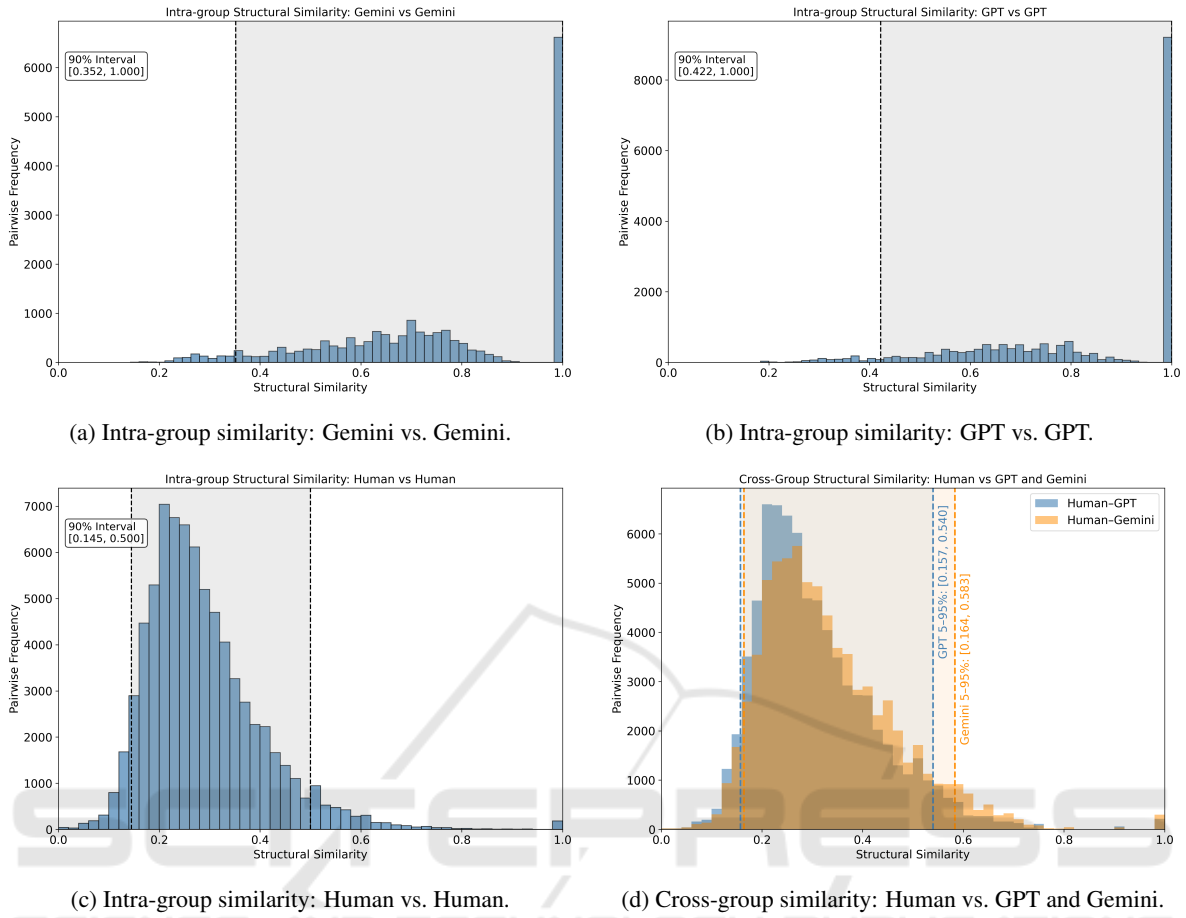


Figure 1: Distribution of structural similarity scores computed using normalized subtree hashing over Abstract Syntax Trees.

Intra-Group Similarity. Both LLMs exhibit profound structural convergence. For GPT-5.2 (Fig. 1b), 90% of pairwise similarities fall within [0.422, 1.000]. Notably, 82.32% of intra-GPT comparisons exceed a 0.60 threshold, and nearly half (48.47%) are structurally identical ($J = 1.00$). While Gemini 3.0 Flash solutions (Fig. 1a) demonstrate slightly greater variability, they remain highly deterministic: 74.49% exceed the 0.60 threshold and over a third (34.83%) collapse to identical normalized representations.

In contrast, human-written solutions (Fig. 1c) display markedly broader structural diversity. The 90% interval for human-human comparisons is [0.145, 0.500]. Only 1.56% of human-human pairs exceed 0.60 similarity, and just 0.24% are structurally identical (Table 1). Exact structural convergence is therefore over two orders of magnitude more frequent among GPT-generated solutions than among human-authored implementations.

Cross-Group Similarity. Panel (d) presents the cross-group structural similarity between human-

authored solutions and those generated by GPT-5.2 and Gemini 3.0 Flash. The 5th–95th percentile interval for Human–GPT comparisons spans [0.157, 0.540], while Human–Gemini comparisons span [0.164, 0.583]. High-similarity cases remain exceedingly rare: only 2.16% (Human–GPT) and 3.91% (Human–Gemini) of comparisons exceed a 0.60 threshold, and fewer than 0.50% exceed 0.90 (Table 1). Exact structural matches between human and LLM solutions occur in fewer than 0.40% of cases. Crucially, these cross-group distributions closely resemble the human intra-group baseline and remain starkly separated from the highly concentrated intra-LLM distributions. Even at extreme thresholds, cross-group similarity never approaches the structural determinism observed within the AI cohorts.

Synthesis of Structural Trends. Taken together, these data reveal three distinct structural paradigms. Human intra-group comparisons exhibit broad dispersion with virtually no identical structures, while cross-group (Human–LLM) comparisons largely mir-

Table 2: Comment similarity statistics across solution sources, computed using TF-IDF cosine similarity.

Comparison	Mean	Median	5–95% Interval	>0.60	>0.90	= 1.00
Human–Human	0.088	0.035	[0.000, 0.325]	1.76%	1.03%	0.45%
GPT–GPT	0.836	1.000	[0.177, 1.000]	78.81%	71.59%	44.03%
Gemini–Gemini	0.445	0.427	[0.100, 0.880]	23.44%	4.39%	1.49%
Human–GPT	0.116	0.022	[0.000, 0.678]	6.23%	1.55%	0.00%
Human–Gemini	0.116	0.073	[0.000, 0.375]	2.06%	0.95%	0.39%

ror this baseline, showing only moderate overlap. In severe contrast, LLM intra-group distributions demonstrate extreme concentration and frequent structural cloning. This persistent and pronounced gap between cross-group and intra-LLM similarity provides strong empirical evidence that LLM-generated and human-written solutions occupy fundamentally distinct regions of the algorithmic solution space. Additionally, cross-model comparisons reveal that AI-generated solution spaces are not uniform across models, but instead consist of partially overlapping clusters with varying degrees of structural convergence.

4.2 Comment Similarity Analysis

Beyond executable logic, the lexical similarity of code comments was analyzed using TF-IDF representations and pairwise cosine similarity, denoted as $Sim_{cos}(p_i, p_j)$. This metric captures overlap in explanatory language while remaining robust to minor lexical variations. As detailed in Table 2, the resulting distributions reveal a pronounced divergence between human- and LLM-generated commentary.

Intra-Group Similarity. Human-authored solutions exhibit low lexical overlap, with a mean similarity of 0.088, a median of 0.035, and a 5th–95th percentile interval of [0.000, 0.325]. High-similarity cases are exceedingly rare (only 1.76% exceed a 0.60 threshold, and 0.45% are exact matches), indicating substantial stylistic heterogeneity.

In contrast, GPT-generated comments display extreme convergence (mean = 0.836; median = 1.000). Notably, 78.81% of GPT pairs exceed the 0.60 similarity threshold, and 44.03% are exact matches ($Sim_{cos} = 1.000$). The median value of 1.000 implies that over half of all intra-GPT comment pairs are effectively identical, a finding consistent with highly deterministic, templated generation.

Gemini 3.0 Flash solutions exhibit intermediate convergence (mean = 0.445; median = 0.427). While 23.44% of Gemini pairs exceed the 0.60 threshold, only 1.49% are identical. This suggests that Gem-

ini 3.0 Flash produces greater explanatory variability than GPT-5.2, yet still maintains markedly higher homogeneity than the human baseline.

Cross-Group Similarity. Cross-group comparisons further illuminate this stylistic separation. Both Human–GPT and Human–Gemini comparisons yield remarkably low similarities (means of 0.116 in both cases), closely mirroring the Human–Human baseline. Exact matches are virtually non-existent in cross-group settings (0.00% for Human–GPT and 0.39% for Human–Gemini). These findings confirm that LLM comment distributions form a lexically cohesive cluster operating largely distinctly from human-authored commentary.

Synthesis of Lexical Trends. Taken together, these results demonstrate severe lexical convergence among LLM-generated solutions contrasting sharply with the profound stylistic diversity of human programmers. Whereas human commentary reflects highly individualized explanatory habits, LLMs frequently reproduce near-identical descriptive patterns across independent generations. This lexical determinism strongly complements the previously observed structural convergence, firmly establishing comment similarity as a robust and highly discriminative feature for the detection of synthetic code.

4.3 Identifier Diversity

To evaluate naming variability, we compute Shannon entropy over user-defined identifiers within each solution and measure cross-solution identifier similarity using Jaccard overlap. Entropy captures intra-solution lexical diversity, while Jaccard similarity reflects reuse of identical naming conventions across independent implementations.

Identifier Entropy. Table 4 summarizes the within-solution entropy statistics ($H(p)$). Contrary to the structural and comment-level findings, LLM-generated solutions do not exhibit reduced lexical

Table 3: Cross-group identifier Jaccard similarity (Human vs AI).

Comparison	Mean	Median	95th pct	>0.60	>0.90
Human–GPT	0.294	0.300	0.667	7.85%	1.03%
Human–Gemini	0.307	0.308	0.714	9.88%	1.37%
Human–AI (Combined)	0.300	0.308	0.714	8.87%	1.20%

diversity at the variable level. GPT-5.2 and Gemini 3.0 Flash display higher mean identifier entropy ($H(p) = 3.024$ and 2.944 , respectively) compared to human solutions (2.136). Moreover, the AI entropy distributions show substantially lower variance ($SD \approx 0.47$) than the human cohort ($SD = 1.30$), indicating a highly consistent naming complexity across generated outputs.

The human distribution, by contrast, is markedly more dispersed. While some human programmers employ rich and varied naming, many rely on minimal or repetitive identifiers typical of algorithmic problem-solving (e.g., single-letter variables), producing a heavily long-tailed entropy distribution. In contrast, LLMs consistently employ moderately rich and descriptive naming patterns, suggesting a standardized, albeit non-simplified, lexical behavior.

Table 4: Identifier entropy statistics across solution sources.

Group	Mean	Median	Std
Human	2.136	2.646	1.300
GPT	3.024	3.000	0.478
Gemini	2.944	2.914	0.457
AI (Combined)	2.984	2.950	0.470

Cross-Solution Identifier Overlap. Cross-group identifier similarity (J_{id}) remains moderate (Table 3). Human–AI Jaccard similarity averages approximately 0.30 , with fewer than 9% of pairs exceeding the 0.60 similarity threshold and roughly 1% exceeding 0.90 . These values indicate partial convergence toward canonical, task-specific identifiers (e.g., conventional algorithmic variable names), but without the extreme clustering observed in the structural or comment analyses.

Importantly, identifier overlap does not exhibit the pronounced inter-cohort separation seen in the structural similarity analysis. While both GPT-5.2 and Gemini 3.0 Flash demonstrate standardized naming conventions, they do not collapse onto near-identical identifier sets across independent solutions. Instead, their identifier selection appears constrained yet non-deterministic.

Synthesis of Identifier Trends. Taken together, the identifier analysis suggests a standardization effect rather than outright lexical simplification in LLM-generated code. Compared to the profound structural and comment-level convergence observed earlier, identifier diversity provides a weaker, yet highly complementary, discriminative signal. LLM-generated solutions systematically exhibit stable and moderately rich naming patterns, whereas human-authored solutions display substantially greater variability and idiosyncrasy in their lexical behavior.

5 DISCUSSION

The results demonstrate that LLM-generated code exhibits systematic regularities fundamentally absent in human-authored solutions. While GPT-5.2 produces highly deterministic, structurally cloned outputs, and Gemini 3.0 Flash shows slightly greater variability, both models generate significantly more concentrated solution distributions than humans. This indicates that the primary detection signal lies in the divergence between human and AI-generated solution spaces, rather than in distinguishing between specific models.

From an educational perspective, the goal of similarity-based analysis is not to penalize iterative, AI-assisted learning, but to identify zero-effort submissions—cases where students generate a solution in a zero-shot manner and submit it without comprehension. Under such conditions, LLM outputs tend to form compact, model-specific clusters, leaving a distinct multidimensional fingerprint.

A potential limitation is the role of prompt design. More sophisticated prompts may increase solution diversity; however, this study intentionally models zero-shot usage, where minimal instructions are provided without iterative refinement. This reflects realistic misuse scenarios in introductory programming contexts, suggesting that the observed convergence is characteristic of typical usage rather than a prompting artifact.

Importantly, high structural similarity alone does not prove misconduct, as algorithmic problems admit canonical solutions. To mitigate false positives, educators can construct a synthetic reference corpus by generating N independent zero-shot solutions for

a given task. Because such outputs cluster tightly, this reference set approximates the model's solution space. Submissions exhibiting extreme multi-layer overlap with this cluster provide a strong probabilistic signal of unedited AI generation.

This approach is conceptually aligned with recent work on pseudo-AI reference submissions for detection (Bashir, 2025), although our results emphasize the role of structural clustering as the primary discriminative signal.

6 CONCLUSION

This study evaluated the multidimensional properties of human-written and LLM-generated algorithmic solutions. The findings empirically demonstrate that LLM-generated code collapses into compact, model-specific clusters, whereas human code inherently resists such standardization.

Ultimately, while no single metric can definitively identify synthetic code, the intersection of structural and lexical convergence provides a highly discriminative, evidence-based foundation for probabilistic authorship attribution in computer science education.

REFERENCES

- Ambati, S. H., Stakhanova, N., and Branca, E. (2023). Learning ai coding style for software plagiarism detection. In *International Conference on Security and Privacy in Communication Systems*, pages 467–489. Springer.
- Bashir, S. (2025). Using pseudo-ai submissions for detecting ai-generated code. *Frontiers in Computer Science*, Volume 7 - 2025.
- Bisztray, T., Cherif, B., Dubniczky, R. A., Gruschka, N., Borsos, B., Ferrag, M. A., Kovacs, A., Mavroeidis, V., and Tihanyi, N. (2026). I know which llm wrote your code last summer: Llm generated code stylometry for authorship attribution. In *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security, AISec '25*, page 28–39, New York, NY, USA. Association for Computing Machinery.
- Cotroneo, D., Improtà, C., and Liguori, P. (2025). Human-written vs. ai-generated code: A large-scale study of defects, vulnerabilities, and complexity.
- Dzialets, K., Makeeva, A., Vlasov, I., Potriasaeva, A., Rostovskii, A., Golubev, Y., and Birillo, A. (2025). Everything you need to know about cs education: Open results from a survey of more than 18,000 participants. In *Proceedings of the ACM Global on Computing Education Conference 2025 Vol 1*, pages 50–56.
- Ellis, A., Jones, B., and Smith, C. (2024). Chatgpt and python programming homework. *Decision Sciences Journal of Innovative Education*, 22(3):123–145.
- Gacek, P. (2025). An embedding-based approach for identifying LLM-generated code in student assignments. In *Proceedings of the GeCoIn 2025: Generative Code Intelligence Workshop*, volume 4075 of *CEUR Workshop Proceedings*. CEUR-WS.org. Co-located with the 28th European Conference on Artificial Intelligence (ECAI-2025).
- Google DeepMind (2026). Gemini 3.0 flash: A high-speed, efficient large multimodal model. <https://blog.google/products-and-platforms/products/gemini/gemini-3-flash/>. Accessed: 2026-02-16.
- Li, R., Li, M., and Qiao, W. (2025). Engineering students' use of large language model tools: An empirical study based on a survey of students from 12 universities. *Education Sciences*, 15(3).
- OpenAI (2022). Introducing ChatGPT. <https://openai.com/index/chatgpt/>. Accessed: 2026-02-20.
- OpenAI (2026). Introducing gpt-5.0: Our most capable and aligned model. <https://openai.com/index/introducing-gpt-5-0/>. Accessed: 2026-02-16.
- Prechelt, L. and Malpohl, G. (2003). Finding plagiarisms among a set of programs with jplag. *Journal of Universal Computer Science*, 8.
- Sağlam, T. (2025). *Mitigating Automated Obfuscation Attacks on Software Plagiarism Detection Systems*. PhD thesis, Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany. Doctoral dissertation.
- Schleimer, S., Wilkerson, D. S., and Aiken, A. (2003). Windowing: Local algorithms for document fingerprinting. In *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, pages 76–85.
- Shi, Y., Zhang, H., Wan, C., and Gu, X. (2025). *Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers*, page 1628–1639. IEEE Press.
- Suh, H., Tafreshipour, M., Li, J., Bhattiprolu, A., and Ahmed, I. (2024). An empirical study on automatically detecting ai-generated source code: How far are we? *arXiv preprint arXiv:2411.04299*.
- Yan, L., McKeown, N., Sahami, M., and Piech, C. (2018). Tmoss: Using intermediate assignment work to understand excessive collaboration in large classes. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education, SIGCSE '18*, page 110–115, New York, NY, USA. Association for Computing Machinery.
- Ye, J., Chen, X., Xu, N., Zu, C., Shao, Z., Liu, S., Cui, Y., Zhou, Z., Gong, C., Shen, Y., et al. (2023). A comprehensive capability analysis of gpt-3 and gpt-3.5 series models. *arXiv preprint arXiv:2303.10420*.
- Zibada (2023). Google code jam archive. Unofficial archive of Google Code Jam competitions (2003–2022). Accessed: 2025-01-29.